Norwegian reservoirs and cross-border electricity transfers

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

sns.set_theme(style="whitegrid")
```

NVE API retrieval and preparation

Retrieve public observations and inspect their coverage.

```
from ind320.data.reservoirs import (
    INTERVAL_NOTICE,
    load_area_metadata,
    load_reservoir_data,
    unusual_fill_levels,
)


start_month, end_month, area_type = None, None, None
df, fill_levels = load_reservoir_data(start_month, end_month, area_type)
print(INTERVAL_NOTICE)
print(f"{len(df):,} records; {len(fill_levels.columns)} observed area series")
display(df.head())
```

```
2026-10-18 18:47:02.881 No runtime found, using MemoryCacheStorageManager
2026-10-18 18:47:02.923 No runtime found, using MemoryCacheStorageManager
2026-10-18 18:47:02.931 No runtime found, using MemoryCacheStorageManager
2026-10-18 18:47:02.967 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when
running in bare mode.

NVE's public endpoint returns the full history. The app caches that download for one hour and displays t
he selected months. The API has no date filters, so the month slider cannot limit the download.
14,922 records; 9 observed area series
```

	Observation Date	Area Type	Area Code	Iso Year	Iso Week	Fill Level	Capacity TWh	Stored Energy TWh	Next Publication Date	Previous Week Fill Level	Weekly Fill Level Change
0	1995-01-08	EL	1	1995	1	0.617574	6.003264	3.707457	NaT	0.583705	0.033869
1	1995-01-08	EL	2	1995	1	0.630947	34.043590	21.479713	NaT	0.777604	-0.146657
2	1995-01-08	EL	3	1995	1	0.602376	8.920932	5.373754	NaT	0.596027	0.006349
3	1995-01-08	EL	4	1995	1	0.558864	21.079208	11.780413	NaT	0.801348	-0.242484
4	1995-01-08	EL	5	1995	1	0.616448	17.390587	10.720388	NaT	0.614288	0.002160

Area coverage and unusual readings

```
# Join counts to all metadata areas so an unobserved area remains visible.
areas = load_area_metadata()
counts = df.groupby(["Area Type", "Area Code"]).size().rename("Observation count")
area_coverage = areas.join(counts, on=["Area Type", "Area Code"])
area_coverage["Observation count"] = (
    area_coverage["Observation count"].fillna(0).astype(int)
)
display(area_coverage)
unusual = unusual_fill_levels(df)
print(f"{len(unusual)} readings outside 0–100%; original values preserved.")
display(unusual)
```

```
2026-10-18 18:47:05.167 No runtime found, using MemoryCacheStorageManager
```

	Area Type	Area Code	Name	Long Name	Description	Observation count
0	EL	1	NO 1	Elspotområde 1	Sørøst-Norge. Omfatter østlige del av Østlande...	1658
1	EL	2	NO 2	Elspotområde 2	Sørvest-Norge. Omfatter mesteparten av Vestfol...	1658
2	EL	3	NO 3	Elspotområde 3	Midt-Norge. Omfatter nordre og vestlige del av...	1658
3	EL	4	NO 4	Elspotområde 4	Nord-Norge. Omfatter resten av Trøndelag og No...	1658
4	EL	5	NO 5	Elspotområde 5	Vest-Norge. Omfatter midtre del av Vestland op...	1658
5	NO	0	Norge	Norge	Hele landet	1658
6	VASS	1	VASS1	Vassdragsområde 1	Sørøst-Norge. Østlandet, Agder og deler av Rog...	1658
7	VASS	2	VASS2	Vassdragsområde 2	Vest-Norge. Resten av Rogaland og Vestland.	1658
8	VASS	3	VASS3	Vassdragsområde 3	Midt-Norge. Møre og Romsdal, Trøndelag og sørl...	1658
9	VASS	4	VASS4	Vassdragsområde 4	Nord-Norge. Resten av Nordland og nordover.	0

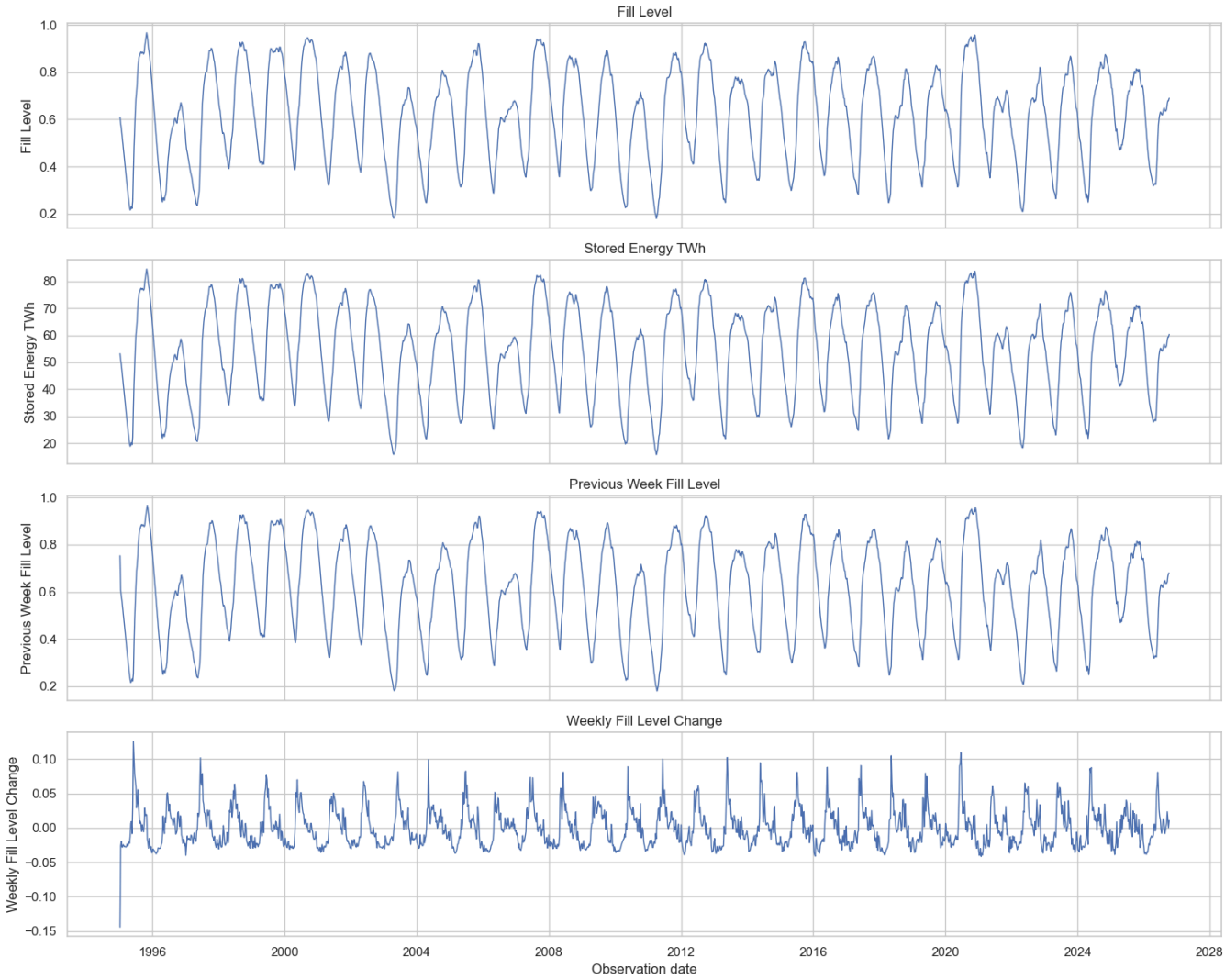
```
3 readings outside 0–100%; original values preserved.
```

	Observation Date	Area Type	Area Code	Iso Year	Iso Week	Fill Level	Capacity TWh	Stored Energy TWh	Next Publication Date	Previous Week Fill Level	Weekly Fill Level Change
2718	2000-10-22	EL	1	2000	42	1.003412	6.003264	6.023747	NaT	0.996905	0.006507
13428	2023-08-13	EL	1	2023	32	1.020072	6.003264	6.123758	2023-08-23 13:00:00	0.894086	0.125986
13446	2023-08-27	EL	1	2023	34	1.014961	6.003264	6.093078	2023-09-06 13:00:00	0.997816	0.017145

National measurements as separate time series

```
national = df.loc[(df["Area Type"] == "NO") & (df["Area Code"] == 0)].copy()
national = national.sort_values("Observation Date")
measurement_columns = [
    "Fill Level",
    "Stored Energy TWh",
    "Previous Week Fill Level",
    "Weekly Fill Level Change",
]

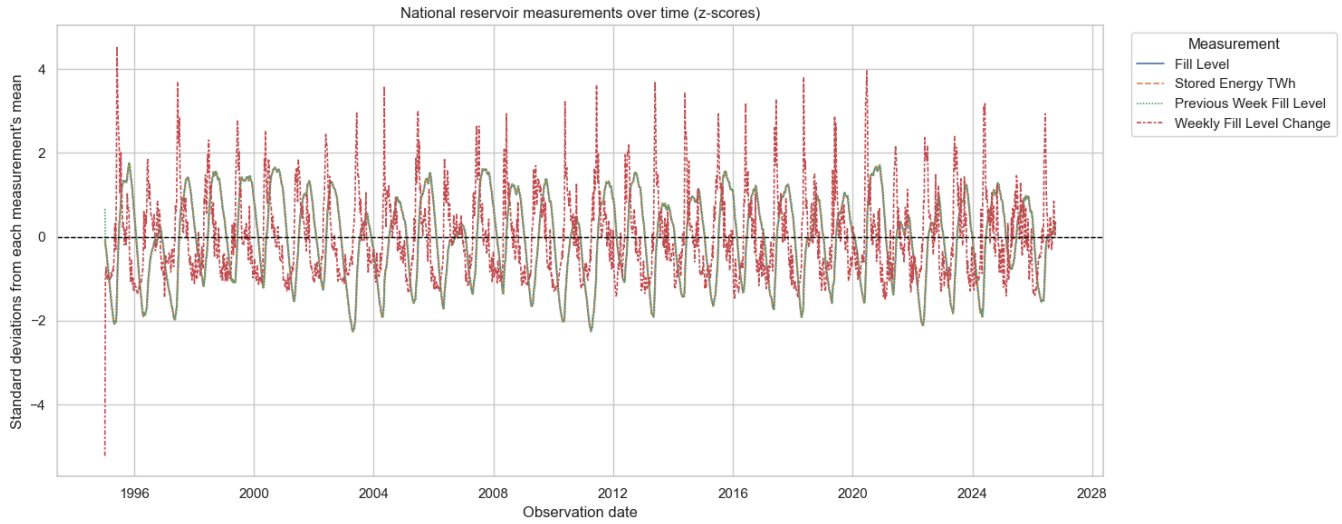
# Plot each original measurement on its own axis.
fig, axes = plt.subplots(len(measurement_columns), 1, figsize=(15, 12), sharex=True)
for ax, column in zip(axes, measurement_columns):
    sns.lineplot(
        data=national,
        x="Observation Date",
        y=column,
        estimator=None,
        errorbar=None,
        linewidth=1,
        ax=ax,
    )
    ax.set_title(column)
    ax.set_xlabel("")
axes[-1].set_xlabel("Observation date")
fig.tight_layout()
plt.show()
```


Fill Level
Fill Level
1.0
0.8
0.6
0.4
0.2
Stored Energy TWh
Stored Energy TWh
80
70
60
50
40
30
20
Previous Week Fill Level
Previous Week Fill Level
1.0
0.8
0.6
0.4
0.2
Weekly Fill Level Change
Weekly Fill Level Change
0.10
0.05
0.00
−0.05
−0.10
−0.15
1996
2000
2004
2008
2012
2016
2020
2024
2028
Observation date

The same time series together on a standardised scale

```
# Reuse the four series above, standardising each across its own dates.
standardised = (
    national[measurement_columns] - national[measurement_columns].mean()
) / national[measurement_columns].std()
standardised["Observation Date"] = national["Observation Date"].to_numpy()
comparison_data = standardised.melt(
    id_vars="Observation Date", var_name="Measurement", value_name="Z-score"
)

fig, ax = plt.subplots(figsize=(15, 6))
sns.lineplot(
    data=comparison_data,
    x="Observation Date",
    y="Z-score",
    hue="Measurement",
    style="Measurement",
    estimator=None,
    errorbar=None,
    linewidth=1.2,
    ax=ax,
)
ax.axhline(0, color="black", linewidth=1, linestyle="--")
ax.set_title("National reservoir measurements over time (z-scores)")
ax.set_xlabel("Observation date")
ax.set_ylabel("Standard deviations from each measurement's mean")
ax.legend(title="Measurement", bbox_to_anchor=(1.02, 1), loc="upper left")
fig.tight_layout()
plt.show()
```



ENTSO-E API retrieval: 2024 and 2025

```
from pathlib import Path

from ind320.data.transfers import ingest, prepare_intervals

refresh_api = False
snapshot = Path(".local/transfers.pkl")
coverage_snapshot = Path(".local/coverage.pkl")
if refresh_api or not (snapshot.exists() and coverage_snapshot.exists()):
    observations, coverage = ingest((2024, 2025))
    snapshot.parent.mkdir(exist_ok=True)
    observations.to_pickle(snapshot)
    coverage.to_pickle(coverage_snapshot)
    print("Retrieved live ENTSO-E data.")
else:
    # Read only trusted local snapshots produced by this project.
    observations = pd.read_pickle(snapshot)
    coverage = pd.read_pickle(coverage_snapshot)
    print("Reused local snapshot from the previous live ENTSO-E retrieval.")
print(
    "Snapshot saved (UTC):", pd.Timestamp(snapshot.stat().st_mtime, unit="s", tz="UTC")
)
observations = prepare_intervals(observations)
print(
    f"{len(observations):,} intervals; {coverage.complete.sum()} / {len(coverage)} complete direction-mo
nths"
)
display(coverage.loc[~coverage.complete])
display(observations.head())

# List the external zone pairs explicitly, without overlapping country totals.
from ind320.data.transfers import CONNECTIONS, ZONES

display(
    pd.DataFrame(
        [
            {
                "Connection": label,
                "Norwegian EIC": ZONES[no],
                "Foreign EIC": ZONES[foreign],
            }
            for label, (no, foreign) in sorted(CONNECTIONS.items())
        ]
    )
)
```

```
Reused local snapshot from the previous live ENTSO-E retrieval.
Snapshot saved (UTC): 2026-10-17 14:37:10.493925571+00:00
361,113 intervals; 432 / 480 complete direction-months
```

	connection	year	month	direction	observed_hours	expected_hours	complete	message
432	NO4–RU	2024	1	import	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
433	NO4–RU	2024	2	import	0.0	696.0	False	ENTSO-E reports unavailable data or denied acc...

	connection	year	month	direction	observed_hours	expected_hours	complete	message
434	NO4–RU	2024	3	import	0.0	743.0	False	ENTSO-E reports unavailable data or denied acc...
435	NO4–RU	2024	4	import	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
436	NO4–RU	2024	5	import	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
437	NO4–RU	2024	6	import	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
438	NO4–RU	2024	7	import	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
439	NO4–RU	2024	8	import	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
440	NO4–RU	2024	9	import	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
441	NO4–RU	2024	10	import	0.0	745.0	False	ENTSO-E reports unavailable data or denied acc...
442	NO4–RU	2024	11	import	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
443	NO4–RU	2024	12	import	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
444	NO4–RU	2024	1	export	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
445	NO4–RU	2024	2	export	0.0	696.0	False	ENTSO-E reports unavailable data or denied acc...
446	NO4–RU	2024	3	export	0.0	743.0	False	ENTSO-E reports unavailable data or denied acc...
447	NO4–RU	2024	4	export	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
448	NO4–RU	2024	5	export	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
449	NO4–RU	2024	6	export	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
450	NO4–RU	2024	7	export	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
451	NO4–RU	2024	8	export	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
452	NO4–RU	2024	9	export	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
453	NO4–RU	2024	10	export	0.0	745.0	False	ENTSO-E reports unavailable data or denied acc...
454	NO4–RU	2024	11	export	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
455	NO4–RU	2024	12	export	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
456	NO4–RU	2025	1	import	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
457	NO4–RU	2025	2	import	0.0	672.0	False	ENTSO-E reports unavailable data or denied acc...
458	NO4–RU	2025	3	import	0.0	743.0	False	ENTSO-E reports unavailable data or denied acc...
459	NO4–RU	2025	4	import	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
460	NO4–RU	2025	5	import	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...

	connection	year	month	direction	observed_hours	expected_hours	complete	message
461	NO4–RU	2025	6	import	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
462	NO4–RU	2025	7	import	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
463	NO4–RU	2025	8	import	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
464	NO4–RU	2025	9	import	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
465	NO4–RU	2025	10	import	0.0	745.0	False	ENTSO-E reports unavailable data or denied acc...
466	NO4–RU	2025	11	import	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
467	NO4–RU	2025	12	import	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
468	NO4–RU	2025	1	export	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
469	NO4–RU	2025	2	export	0.0	672.0	False	ENTSO-E reports unavailable data or denied acc...
470	NO4–RU	2025	3	export	0.0	743.0	False	ENTSO-E reports unavailable data or denied acc...
471	NO4–RU	2025	4	export	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
472	NO4–RU	2025	5	export	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
473	NO4–RU	2025	6	export	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
474	NO4–RU	2025	7	export	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
475	NO4–RU	2025	8	export	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...
476	NO4–RU	2025	9	export	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
477	NO4–RU	2025	10	export	0.0	745.0	False	ENTSO-E reports unavailable data or denied acc...
478	NO4–RU	2025	11	export	0.0	720.0	False	ENTSO-E reports unavailable data or denied acc...
479	NO4–RU	2025	12	export	0.0	744.0	False	ENTSO-E reports unavailable data or denied acc...

	connection	direction	start	end	power_mw	energy_mwh
0	NO1–SE3	export	2023-12-31 23:00:00+00:00	2024-01-02 16:00:00+00:00	0.00	0.00
1	NO1–SE3	export	2024-01-02 16:00:00+00:00	2024-01-02 17:00:00+00:00	27.83	27.83
2	NO1–SE3	export	2024-01-02 17:00:00+00:00	2024-01-03 17:00:00+00:00	0.00	0.00
3	NO1–SE3	export	2024-01-03 17:00:00+00:00	2024-01-03 18:00:00+00:00	35.19	35.19
4	NO1–SE3	export	2024-01-03 18:00:00+00:00	2024-01-04 06:00:00+00:00	0.00	0.00

	Connection	Norwegian EIC	Foreign EIC
0	NO1–SE3	10YNO-1--------2	10Y1001A1001A46L
1	NO2–DE_LU	10YNO-2--------T	10Y1001A1001A82H
2	NO2–DK1	10YNO-2--------T	10YDK-1--------W
3	NO2–GB	10YNO-2--------T	10YGB----------A
4	NO2–NL	10YNO-2--------T	10YNL----------L

	Connection	Norwegian EIC	Foreign EIC
5	NO3–SE2	10YNO-3--------J	10Y1001A1001A45N
6	NO4–FI	10YNO-4--------9	10YFI-1--------U
7	NO4–RU	10YNO-4--------9	10Y1001A1001A49F
8	NO4–SE1	10YNO-4--------9	10Y1001A1001A44P
9	NO4–SE2	10YNO-4--------9	10Y1001A1001A45N

Spark insertion into Cassandra and Spark extraction

```
# Store API data, extract relevant columns, and check the round trip.
from ind320.storage.pipeline import local_databases, read_cassandra, write_cassandra

with local_databases() as (spark, keyspace):
    write_cassandra(spark, keyspace, observations)
    extracted = read_cassandra(spark, keyspace)
    pd.testing.assert_frame_equal(observations, extracted, check_dtype=False)
print(f"Spark/Cassandra round trip passed: {len(extracted):,} intervals.")
```

```
[Stage 0:>                                                          (0 + 2) / 2]
```

Publish curated intervals and annual summaries to MongoDB

```
from ind320.storage.mongo_store import available_transfers, publish, query_annual, query_months

summaries = publish(extracted)
catalog = available_transfers()
print(f"Published {len(extracted):,} intervals and {len(summaries)} annual summaries.")
display(summaries)
```

```
Published 361,113 intervals and 40 annual summaries.
```

	connection	year	direction	energy_mwh	observed_hours	expected_hours	complete
0	NO1–SE3	2024	import	5.389376e+06	8784.0	8784.0	True
1	NO1–SE3	2024	export	1.505030e+06	8784.0	8784.0	True
2	NO1–SE3	2025	import	6.140554e+06	8760.0	8760.0	True
3	NO1–SE3	2025	export	1.623833e+06	8760.0	8760.0	True
4	NO2–DE_LU	2024	import	1.218802e+06	8784.0	8784.0	True
5	NO2–DE_LU	2024	export	6.998108e+06	8784.0	8784.0	True
6	NO2–DE_LU	2025	import	7.674199e+05	8760.0	8760.0	True
7	NO2–DE_LU	2025	export	7.787682e+06	8760.0	8760.0	True
8	NO2–DK1	2024	import	1.652879e+06	8784.0	8784.0	True
9	NO2–DK1	2024	export	8.119131e+06	8784.0	8784.0	True
10	NO2–DK1	2025	import	1.153915e+06	8760.0	8760.0	True
11	NO2–DK1	2025	export	8.157331e+06	8760.0	8760.0	True
12	NO2–GB	2024	import	2.733144e+05	8784.0	8784.0	True
13	NO2–GB	2024	export	1.018869e+07	8784.0	8784.0	True
14	NO2–GB	2025	import	4.502173e+05	8760.0	8760.0	True
15	NO2–GB	2025	export	9.763524e+06	8760.0	8760.0	True
16	NO2–NL	2024	import	5.961079e+05	8784.0	8784.0	True
17	NO2–NL	2024	export	3.496670e+06	8784.0	8784.0	True
18	NO2–NL	2025	import	4.157648e+05	8760.0	8760.0	True
19	NO2–NL	2025	export	2.791009e+06	8760.0	8760.0	True
20	NO3–SE2	2024	import	3.023801e+06	8784.0	8784.0	True
21	NO3–SE2	2024	export	4.760895e+05	8784.0	8784.0	True
22	NO3–SE2	2025	import	1.290701e+06	8760.0	8760.0	True
23	NO3–SE2	2025	export	1.090740e+06	8760.0	8760.0	True

	connection	year	direction	energy_mwh	observed_hours	expected_hours	complete
24	NO4–FI	2024	import	3.034370e+04	8784.0	8784.0	True
25	NO4–FI	2024	export	2.566910e+05	8784.0	8784.0	True
26	NO4–FI	2025	import	4.729469e+04	8760.0	8760.0	True
27	NO4–FI	2025	export	2.772307e+05	8760.0	8760.0	True
28	NO4–RU	2024	import	NaN	0.0	8784.0	False
29	NO4–RU	2024	export	NaN	0.0	8784.0	False
30	NO4–RU	2025	import	NaN	0.0	8760.0	False
31	NO4–RU	2025	export	NaN	0.0	8760.0	False
32	NO4–SE1	2024	import	1.425270e+06	8784.0	8784.0	True
33	NO4–SE1	2024	export	7.203888e+05	8784.0	8784.0	True
34	NO4–SE1	2025	import	5.225406e+05	8760.0	8760.0	True
35	NO4–SE1	2025	export	1.294926e+06	8760.0	8760.0	True
36	NO4–SE2	2024	import	5.926346e+05	8784.0	8784.0	True
37	NO4–SE2	2024	export	3.339519e+05	8784.0	8784.0	True
38	NO4–SE2	2025	import	1.360122e+05	8760.0	8760.0	True
39	NO4–SE2	2025	export	5.886327e+05	8760.0	8760.0	True

Annual transferred energy and January physical flows

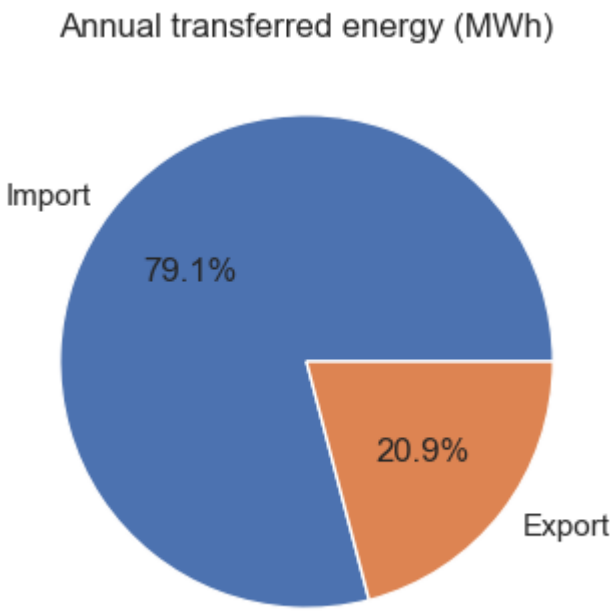
```
from ind320.ui.transfer_charts import energy_pie, transfer_lines
from ind320.data.transfers import clip_intervals, month_bounds

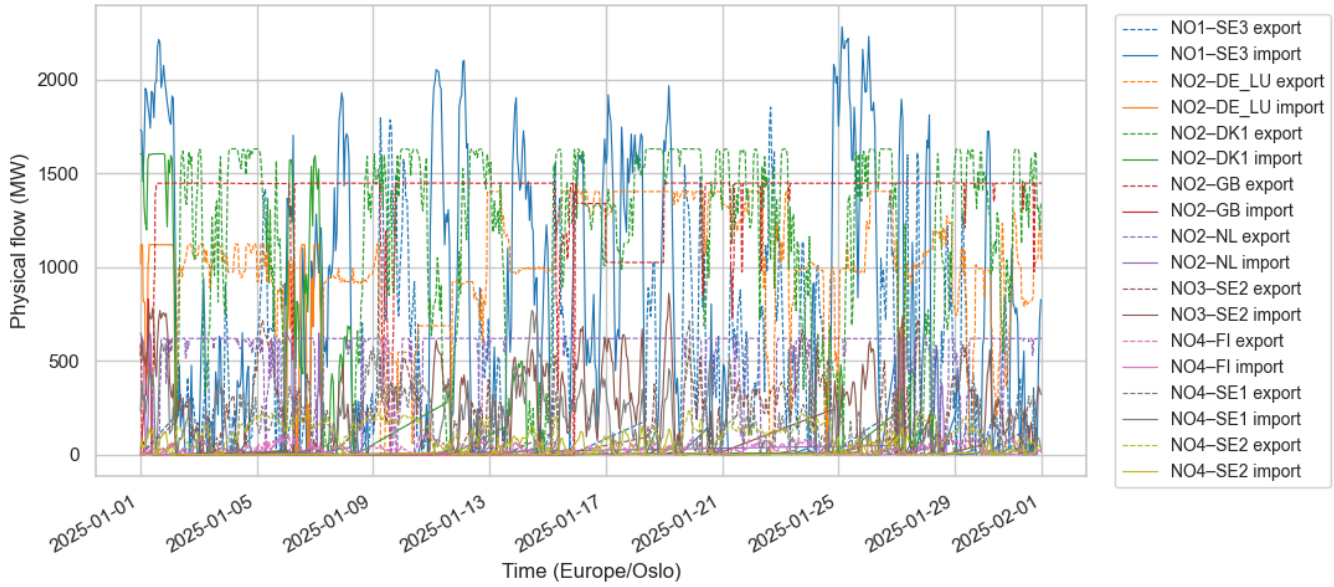
selected_connection = min(catalog.connection.unique())
selected_year = 2025
selected_summary = summaries.loc[
    (summaries.connection == selected_connection) & (summaries.year == selected_year)
]
# Check that the app reads the same annual totals from MongoDB.
selected_summary = selected_summary.sort_values("direction").reset_index(drop=True)
stored_summary = query_annual(selected_connection, selected_year)
pd.testing.assert_frame_equal(
    selected_summary[["direction", "energy_mwh"]],
    stored_summary[["direction", "energy_mwh"]].sort_values("direction").reset_index(drop=True),
    check_dtype=False,
)
display(selected_summary)
fig = energy_pie(selected_summary)
plt.show()
plt.close(fig)

# Compare January on several connections with explicit labels.
january_start, january_end = month_bounds(selected_year, 1)
january = clip_intervals(extracted, january_start, january_end)
fig = transfer_lines(january)
plt.show()
plt.close(fig)

# Read a selected month directly from MongoDB, as the app does.
app_january = query_months(selected_connection, selected_year, [1])
print(f"MongoDB selected-month query: {len(app_january):,} intervals.")
```

	connection	year	direction	energy_mwh	observed_hours	expected_hours	complete
0	NO1–SE3	2025	export	1.623833e+06	8760.0	8760.0	True
1	NO1–SE3	2025	import	6.140554e+06	8760.0	8760.0	True






```
MongoDB selected-month query: 857 intervals.
```

Compulsory work log

For this part of the project, I continued working with Norwegian reservoir levels and electricity transfers. I used Jupyter Notebook to prepare the data, check calculations, and make plots. In Part 1, the reservoir data came from a CSV file. In Part 2, it comes from the NVE API. The notebook and Streamlit app share Python functions, so they use the same calculations.

Working with the NVE data involved handling dates, missing values, and unusual reservoir readings. I kept the unusual readings visible so they can be checked. The app shows reservoir levels as percentages, and area names come from the API. A table also shows which areas have no observations. One issue is that the NVE endpoint returns the full history without a date filter. Streamlit saves that download in its cache and filters the selected months afterwards. This means the slider changes what is shown, but not what is downloaded. This still needs to be clarified with the teacher because of the October 12 requirement.

For electricity transfers, I worked with ENTSO-E data for 2024 and 2025 in both directions across connections to other countries. The calculations convert power in MW to energy in MWh using the length of each interval. Missing data stays as gaps, and the yearly summaries show how many hours are covered. Spark writes the data to Cassandra and reads it back before it is published to MongoDB. The notebook checks the data after these steps, and rerunning the process does not create duplicate records.

In Streamlit, the work is split into Home, Data table, Plots, Transfers, and About pages. Users can choose reservoir areas and months, or explore transfers by connection, year, and month. The transfer page reads from MongoDB. The charts leave gaps where data or selected months are missing. Working with the app also involved handling empty selections and explaining the data beside the charts.

Checks cover date filtering, energy calculations, and the app pages. Private settings are kept in local files or Streamlit secrets. Checking the deployed app is still a separate task to finish. The project is available in the public GitHub repository and the Streamlit app. The project video is available on Google Drive.

AI-tool use

I used AI as support while working on the project. It helped me work through ideas, write and adjust code for the APIs and databases, and add controls in Streamlit. I also used it for help with small tests, simplifying repeated code, and making the notebook text clearer. It helped check the Part 2 requirements and point out the issue with the NVE date filter. I am responsible for checking the code and results, understanding the missing data, and making sure the final submission meets the requirements.